

SMART HOSPITAL QUEUE DAN RECORD SYSTEM

LAPORAN TUGAS ALGORITMA DAN STRUKTUR DASAR

Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, ST, M.AIT.



Disusun Oleh :

Arya Raka Sadewa (25051030074)

Rifat Astradha Putra (25051030066)

Abid Musthofa Anggota (25051030054)

Muh. Abd Rajab. S (25051030059)

PRODI TEKNIK ELEKTRO
DEPARTMENT PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA

2026

DAFTAR ISI

Contents

DAFTAR ISI	2
BAB 1 PENDAHULUAN	5
1.1 Latar Belakang Masalah	5
1.2 Tujuan Sistem.....	6
1.3 Batasan Implementasi	6
1.4 Diagram Arsitektur Sistem.....	7
BAB II PERANCANGAN SISTEM	9
2.1 Pemilihan Struktur Data dan Justifikasi.....	9
1. Priority Queue.....	9
2. Stack.....	9
3. Binary Search Tree (BST)	9
2.2 Diagram Interaksi Antar Modul	10
2.3 Pseudocode Algoritma Utama.....	11
2.3.1 Pseudocode Daftar Pasien	11
2.3.2 Pseudocode Panggil Pasien.....	11
2.3.3 Pseudocode Undo Tindakan.....	12
2.3.4 Pseudocode Rekam Medis	13
2.3.5 Pseudocode Laporan Sistem	13
2.3.6 Pseudocode Simulasi Pasien Random	13
BAB III IMPLEMENTASI.....	15
3.1 Penjelasan Code Per Modul	15
3.1.1 Modul <code>priority_queue.py</code>	15
3.1.2 Modul <code>stack.py</code>	15
3.1.3 Modul Graph (<code>graph.py</code>).....	16
3.1.4 Modul Antrean Pasien (<code>antrean_pasien.py</code>)	16
3.1.5 Modul Dokter Service (<code>dokter_service.py</code>).....	16
3.1.6 Modul Rekam Medis Service (<code>rekam_medis_service.py</code>)	16
3.1.7 Modul Laporan Service (<code>laporan_service.py</code>)	17
3.1.8 Modul Main Program (<code>main.py</code>)	17
3.2 Screenshot CLI Berjalan	17
3.3 Contoh Input/Output Nyata.....	18
3.3.1 Contoh daftar Pasien	18

3.3.2 Contoh Panggil Pasien	18
3.3.3 Contoh Input Tindakan Medis	19
3.3.4 Contoh Rawat Inap / Rawat Jalan	19
3.3.5 Contoh Tampilan Rekam Medis	19
3.3.6 Contoh Undo Tindakan	19
3.3.7 Contoh Cek Rekam Medis Setelah Undo	20
3.3.8 Contoh Laporan Sistem.....	20
3.3.9 Contoh Simulasi Pasien Random.....	20
BAB IV ANALISIS BIG-O	21
4.1 Analisis Kompleksitas Waktu dan Ruang	21
4.1.1 Priority Queue.....	21
4.1.2 Stack.....	21
4.1.3 Binary Search Tree (BST).....	21
4.1.4 Graph.....	22
4.2 Tabel Ringkasan Kompleksitas4.3.1 Teori Big-O.....	22
4.3 Perbandingan Teoritis dan Eksperimen.....	23
BAB V HASIL EKSPERIMEN	24
5.1 Tabel Runtime Untuk 3+ Ukuran Data	24
5.2 Grafik Tren.....	24
5.3 Diskusi Apakah Hasil Prediksi Big-O.....	24
BAB VI JAWABAN PERTANYAAN ANALISIS	26
6.1 Analisis Priority Queue: Linked List vs Heap Berdasarkan Hasil Eksperimen.....	26
Linked List	26
Heap	26
6.2 Analisis BST Rekam Medis Berdasarkan Eksperimen	27
BST Seimbang	27
6.3 Analisis Memori Stack Undo Berdasarkan Sistem	27
6.4 Analisis Sorting Laporan Berdasarkan Eksperimen	28
6.5 Integrasi Graph Berdasarkan Sistem yang Ada	28
• Operasi Relevan	29
BAB VII KESIMPULAN	30
7.1 Ringkasan Pencapaian	30
7.2 Refleksi Trade-off Desain	30
7.3 Saran Pengembangan	31
LAMPIRAN.....	32

Pembagian Tugas Anggota.....	32
Tabel Runtime Lengkap.	32
Log AI	33

BAB 1

PENDAHULUAN

1.1 Latar Belakang Masalah

Pelayanan kesehatan di rumah sakit besar seperti RSUP Dr. Sardjito Yogyakarta menghadapi tantangan signifikan dalam manajemen antrian pasien. Berdasarkan data operasional, rumah sakit ini melayani lebih dari 1.500 kunjungan rawat jalan per hari yang terbagi ke dalam beberapa poliklinik spesialis. Tanpa sistem antrian yang cerdas, pasien dengan kondisi kritis sering kali harus menunggu bersama pasien reguler, yang berpotensi memperburuk kondisi klinis pasien tersebut.

Permasalahan utama yang dihadapi dapat diidentifikasi menjadi tiga aspek. Pertama, sistem antrian konvensional berbasis nomor urut tidak mampu membedakan tingkat kegawatan pasien. Pasien dengan kondisi KRITIS, seperti nyeri dada akut yang berpotensi serangan jantung, mendapatkan perlakuan yang sama dengan pasien REGULER yang datang untuk kontrol rutin. Kedua, pencatatan tindakan medis secara manual rentan terhadap kesalahan input yang sulit dibatalkan atau diperbaiki secara sistematis. Ketiga, pencarian rekam medis dari ratusan ribu data pasien yang tersimpan secara tidak terstruktur membutuhkan waktu yang lama dan tidak efisien.

Ketiga permasalahan tersebut membutuhkan solusi berbasis struktur data yang tepat. Priority Queue berbasis Linked List dapat memastikan pasien KRITIS selalu mendapatkan pelayanan lebih dahulu dengan mekanisme tie-break FIFO untuk pasien dengan prioritas yang sama. Stack digunakan sebagai mekanisme undo tindakan yang dicatat secara salah oleh dokter. Binary Search Tree (BST) dengan kunci nomor rekam medis memungkinkan pencarian rekam medis dengan kompleksitas rata-rata $O(\log n)$, jauh lebih efisien dibandingkan pencarian linier $O(n)$ yang biasa digunakan.

Proyek ini mensimulasikan sistem manajemen antrian rumah sakit yang mengintegrasikan ketiga struktur data tersebut dalam satu sistem CLI terpadu. Simulasi menggunakan dataset sebanyak 500 event acak dengan seed=42 untuk memastikan reproducibility hasil eksperimen, dengan skala pasien 50 hingga 500 per hari yang tersebar di 5 poliklinik.

1.2 Tujuan Sistem

Tujuan dari pembuatan Smart Hospital Queue & Record System adalah sebagai berikut:

- Mengimplementasikan struktur data Queue, Stack, dan Binary Search Tree dalam simulasi sistem rumah sakit.
- Membuat sistem antrian pasien berbasis prioritas agar pasien kritis dapat diproses lebih cepat.
- Mengembangkan sistem penyimpanan riwayat tindakan dokter menggunakan Stack dengan fitur undo.
- Mengimplementasikan pencarian rekam medis menggunakan BST agar proses pencarian data menjadi lebih efisien.
- Menganalisis kompleksitas waktu (Big-O) pada setiap operasi utama yang digunakan dalam sistem.
- Membuat sistem interaktif berbasis CLI untuk simulasi pelayanan rumah sakit secara sederhana.

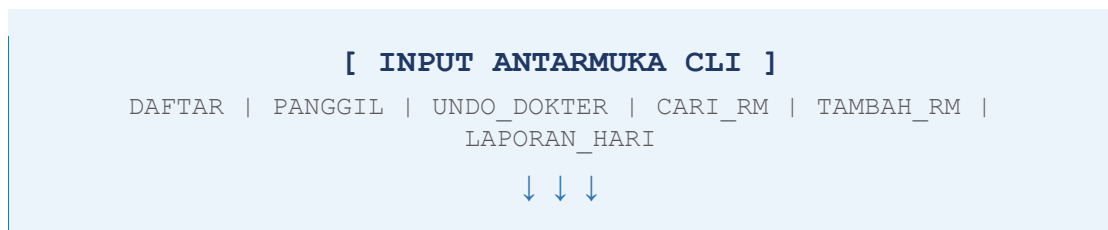
1.3 Batasan Implementasi

Aspek Batasan	Ketentuan
Bahasa Pemrograman	Python 3 tidak menggunakan library struktur data bawaan (collections.deque, heapq.)
Implementasi Struktur Data	Linked List, Stack, Queue, BST wajib diimplementasi dari nol tanpa library eksternal. List Python hanya diizinkan sebagai array biasa.
Jumlah Poliklinik	5 poli : Umum, Jantung, Ortopedi, Anak, dan Gigi — masing-masing memiliki Priority Queue independen.
Tingkat Prioritas	3 level : KRITIS (1), PRIORITAS (2), REGULER (3). Tie-break menggunakan urutan FIFO (waktu pendaftaran).
Dataset & Seed Acak	Seed acak tetap = 42 (np.random.seed(42) dan random.seed (42)) untuk reproducibility eksperimen.

Skala Simulasi	Minimum 50 Maksimum 1000 event campuran per simulasi; eksperimen perbandingan menggunakan $N = 50, 200, 500, 1000$ pasien.
BST Rekam Medis	Menggunakan BST standar. Diskusi degradasi worst-case $O(n)$ disertakan dalam laporan.
Sorting Laporan	Menggunakan Insertion Sort (default) dan Selection Sort implementasi langsung pada Linked List, Big-O $O(n^2)$.
Antarmuka Sistem	CLI interaktif berbasis terminal (Command Line Interface). Tidak mencakup antarmuka grafis (GUI/web).
Ukuran Stack per Dokter	Tidak dibatasi secara eksplisit dalam skenario dasar; analisis memori $O(k)$ di mana k = jumlah tindakan per sesi.

1.4 Diagram Arsitektur Sistem

Diagram berikut menggambarkan hubungan antar modul dan alur data utama dalam sistem:



Modul 1 Priority Queue Multi-Poli 5x Linked List <i>enqueue $O(n)$ dequeue $O(1)$</i>	Modul 2 Stack Riwayat Tindakan Linked List per Dokter <i>push $O(1)$ pop $O(1)$</i>	Modul 3 BST Rekam Medis BST kunci no_rm <i>avg $O(\log n)$ worst $O(n)$</i>
Modul 4 Sorting Laporan Insertion / Selection Sort	Modul 5 CLI Interaktif Entry Point & Orkestrasi	Modul 6 Eksperimen & Validasi

<code>O(n²) pada Linked List</code>	<code>Tampil Big-O per perintah</code>	<code>500 event, seed=42</code> <code>N = 50 200 500</code>
--	--	--

Pasien yang didaftarkan melalui perintah DAFTAR akan masuk ke Priority Queue poli. Setiap tindakan dokter yang dicatat akan di-push ke Stack masing-masing dokter, dan perintah UNDO_DOKTER akan meng-pop tindakan terakhir. Data rekam medis disimpan dan dicari melalui BST. Di akhir sesi, laporan harian diurutkan, sementara Modul 6 menjalankan benchmarking performa secara otomatis dengan seed yang telah ditentukan.

BAB II

PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data dan Justifikasi

Struktur data yang digunakan dalam program ini meliputi Priority Queue, Stack, Binary Search Tree (BST), dan Graph.

1. Priority Queue

Priority Queue digunakan untuk mengatur antrean pasien berdasarkan tingkat prioritas pelayanan. Pada implementasi sistem, prioritas pasien dibagi menjadi tiga kategori, yaitu:

- KRITIS → prioritas tertinggi
- PRIORITAS → prioritas menengah
- REGULER → prioritas biasa

Dengan struktur data ini, proses pengambilan pasien dilakukan berdasarkan prioritas tertinggi sehingga pelayanan menjadi lebih efektif dan realistis sesuai kondisi rumah sakit.

2. Stack

Stack digunakan untuk menyimpan riwayat tindakan dokter sehingga dapat dilakukan proses undo tindakan apabila terjadi kesalahan input atau pembatalan tindakan medis. Konsep yang digunakan adalah LIFO (Last In First Out), yaitu tindakan terakhir yang dimasukkan akan menjadi tindakan pertama yang dihapus. Alasan penggunaan Stack :

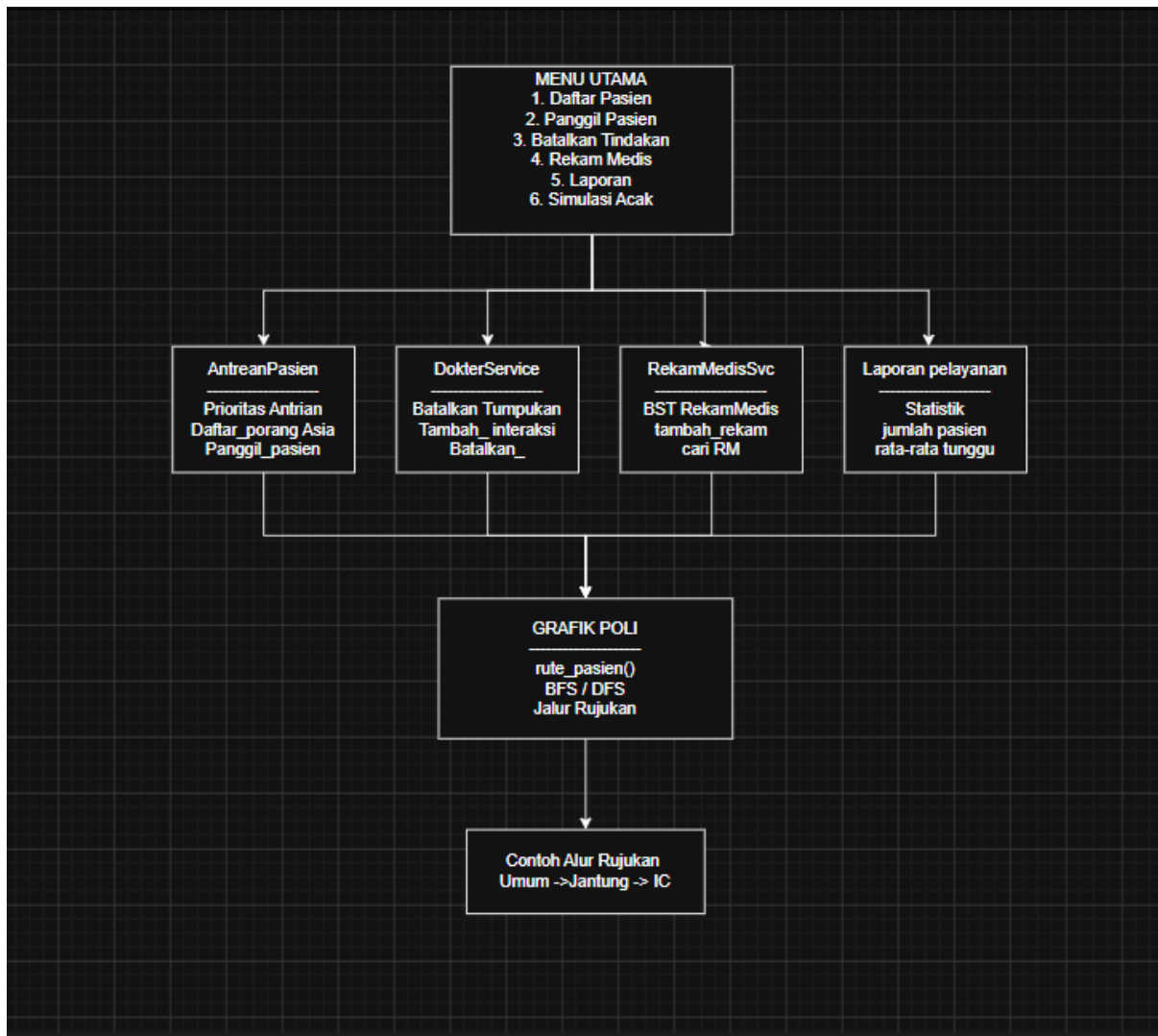
- Mendukung fitur undo tindakan
- Mempermudah koreksi data medis
- Proses push dan pop sangat cepat

3. Binary Search Tree (BST)

BST digunakan untuk menyimpan dan mencari data rekam medis pasien berdasarkan nomor rekam medis. Pada BST, data disusun secara terurut sehingga pencarian data dapat dilakukan lebih cepat dibanding pencarian linear biasa.. Alasan penggunaan BST

- Mempercepat pencarian rekam medis
- Penyimpanan data lebih terstruktur
- Efisien untuk operasi search

2.2 Diagram Interaksi Antar Modul



Penjelasan modul

Modul	Fungsi
Main Menu	Pusat kontrol seluruh sistem
AntreanPasien	Mengatur antrean pasien menggunakan Priority Queue
DokterService	Mengelola tindakan dokter dan fitur undo menggunakan Stack
RekamMedisService	Menyimpan dan mencari rekam medis menggunakan BST
Graph Poli	Mengatur jalur rujukan pasien antar poli
LaporanService	Membuat laporan statistik dan performa sistem

2.3 Pseudocode Algoritma Utama

2.3.1 Pseudocode Daftar Pasien

```
Input nama pasien
Input poli tujuan
Input prioritas pasien

IF poli tersedia THEN
    IF prioritas valid THEN
        Tambahkan pasien ke Priority Queue
        berdasarkan prioritas

        Tampilkan pesan berhasil

        Counter bertambah 1
    ELSE
        Tampilkan "Prioritas tidak valid"
    ENDIF
ELSE
    Tampilkan "Poli tidak tersedia"
ENDIF

END
```

2.3.2 Pseudocode Panggil Pasien

```
Input poli

IF poli tersedia THEN
    Ambil pasien dari Priority Queue

    IF pasien ada THEN
        Tentukan dokter sesuai poli

        Simpan waktu tunggu pasien

        Tampilkan data pasien:
        - Nama
        - Poli
        - Dokter
        - Waktu tunggu

        WHILE tindakan belum selesai
            Input tindakan

            IF tindakan = "selesai" THEN
                BREAK
            ENDIF

            Ambil tanggal dan waktu sekarang

            Simpan tindakan ke Stack

            Simpan rekam medis ke BST

            Tampilkan pesan berhasil
        END WHILE
    ENDIF
ENDIF
```

```

    Cari rekam medis pasien di BST

    IF data ditemukan THEN
        Tampilkan rekam medis
    ENDIF

ELSE

    Tampilkan "Antrean kosong"

ENDIF

ELSE

    Tampilkan "Poli tidak tersedia"

ENDIF

END

```

2.3.3 Pseudocode Undo Tindakan

```

START

Input poli
Input nomor rekam medis

IF poli tersedia THEN

    Cari rekam medis pada BST

    IF rekam medis ditemukan THEN

        Ambil tindakan terakhir dari Stack

        IF tindakan ada THEN

            Hapus riwayat tindakan terakhir

            Tampilkan:
            - Nomor RM
            - Nama pasien
            - Tindakan dihapus
            - Dokter
            - Tanggal

        ELSE

            Tampilkan "Tidak ada tindakan"

        ENDIF

    ELSE

        Tampilkan "Rekam medis tidak ditemukan"

    ENDIF

ELSE

    Tampilkan "Poli tidak tersedia"

ENDIF

END

```

2.3.4 Pseudocode Rekam Medis

```
START
Input nomor rekam medis
Cari data pada BST
IF data ditemukan THEN
    Tampilkan rekam medis pasien
ELSE
    Tampilkan "Rekam medis tidak ditemukan"
ENDIF
END
```

2.3.5 Pseudocode Laporan Sistem

```
START
Tampilkan laporan pasien
Set total = 0
FOR setiap poli
    Tampilkan jumlah pasien per poli
    Tambahkan jumlah pasien ke total
END FOR
Tampilkan total seluruh pasien
Hitung rata-rata waktu tunggu
Tampilkan analisis Big-O
END
```

2.3.6 Pseudocode Simulasi Pasien Random

```
START
Input jumlah pasien random
Generate nama pasien random
Catat waktu mulai
FOR sebanyak jumlah pasien
    Buat nama pasien random
    Pilih poli random
    Pilih prioritas random
    Tambahkan pasien ke antrean
    Counter bertambah
END FOR
```

Catat waktu selesai

Hitung waktu proses

Tampilkan:

- Jumlah pasien berhasil dibuat
- Waktu proses

END

BAB III

IMPLEMENTASI

3.1 Penjelasan Code Per Modul

Pada sistem Smart Hospital Queue and Record System, program dibagi menjadi beberapa modul agar kode lebih terstruktur, mudah dipahami, dan mudah dikembangkan. Berikut penjelasan masing-masing modul yang digunakan.

3.1.1 Modul `priority_queue.py`

```
src/data_structures/priority_queue.py
```

Modul ini digunakan untuk mengatur antrian pasien berdasarkan prioritas pelayanan.

Prioritas pasien:

- KRITIS → prioritas tertinggi
- PRIORITAS → prioritas sedang
- REGULER → prioritas biasa

Fungsi utama:

`enqueue()` → menambahkan pasien

`dequeue()` → mengambil pasien prioritas tertinggi

3.1.2 Modul `stack.py`

```
src/data_structures/stack.py
```

Modul Stack digunakan untuk menyimpan riwayat tindakan dokter menggunakan konsep LIFO (Last In First Out).

Fungsi utama:

`push()` → menambahkan tindakan

`pop()` → menghapus tindakan terakhir

3.1.3 Modul Binary Search Tree (`bst.py`)

Modul `bst.py` digunakan untuk menyimpan dan mencari rekam medis pasien berdasarkan nomor rekam medis. Struktur BST dipilih karena mampu mempercepat proses pencarian data pasien. Saat dokter menambahkan tindakan medis, data rekam medis pasien akan disimpan menggunakan :

```
rekam_medis_service.tambah_rekam_medis(  
    bst_rm,  
    pasien,  
    dokter,  
    tindakan,  
    tanggal  
)
```

3.1.3 Modul Graph (graph.py)

Modul graph.py digunakan untuk menggambarkan hubungan rujukan antar poli rumah sakit. Struktur graph membantu sistem memvisualisasikan jalur layanan pasien.

3.1.4 Modul Antrean Pasien (antrean_pasien.py)

Modul antrean_pasien.py berfungsi untuk mengelola proses pendaftaran dan pemanggilan pasien. Modul ini memiliki fungsi utama :

daftar_pasien()

panggil_pasien()

Saat pasien dipanggil dari antrean :

```
pasien = antrean_service.panggil_pasien(  
    queues[poli]  
)
```

Penjelasan:

Sistem mengambil pasien dengan prioritas tertinggi dari Priority Queue.

Data pasien kemudian dikirim ke proses pemeriksaan dokter.

3.1.5 Modul Dokter Service (dokter_service.py)

Modul dokter_service.py digunakan untuk mengatur dokter dan tindakan medis pasien.

Fungsi utama:

Menentukan dokter sesuai poli

Menambahkan tindakan medis

Melakukan undo Tindakan

3.1.6 Modul Rekam Medis Service (rekam_medis_service.py)

Modul rekam_medis_service.py digunakan untuk menyimpan dan menampilkan rekam medis pasien. Saat tindakan medis ditambahkan, data pasien disimpan ke BST:

```
rekam_medis_service.tambah_rekam_medis(  
    bst_rm,  
    pasien,  
    dokter,  
    tindakan,  
    tanggal  
)
```

- tindakan dokter akan dicatat ke riwayat rekam medis pasien.
- Data yang disimpan meliputi Setiap nama pasien, dokter, tindakan, dan tanggal pemeriksaan.

3.1.7 Modul Laporan Service (laporan_service.py)

Modul laporan_service.py digunakan untuk membuat laporan statistik sistem rumah sakit.

Fungsi utama :

Menampilkan total pasien

Menghitung rata-rata waktu tunggu

Menampilkan analisis Big-O

3.1.8 Modul Main Program (main.py)

Modul main.py merupakan pusat kontrol seluruh sistem Smart Hospital Queue and Record System.

Fungsi utama:

- Menampilkan menu utama
- Mengatur alur program
- Menghubungkan seluruh modul dan struktur data

3.2 Screenshot CLI Berjalan

```
C:\Users\rifat\tpb-asd-Smart-Hospital-
=== SMART HOSPITAL QUEUE SYSTEM ===

Menu:
1. Daftar Pasien
2. Panggil Pasien
3. Undo Tindakan
4. Rekam Medis
5. Laporan
6. Simulasi Pasien Random
7. Keluar
Pilih menu: 1
```

```
Masukkan nomor RM: 1

Undo tindakan berhasil
Tindakan terakhir dibatalkan
Rekam medis berhasil dihapus
Nomor RM 1 bisa digunakan kembali
```

```
Pilih menu: 1
Nama pasien: Rifat
Poli (Umum/Jantung/Ortopedi/Anak/Gigi): Umum
Prioritas (KRITIS/PRIORITAS/REGULER): Kritis
Pasien Rifat berhasil ditambahkan ke poli Umum
Nomor RM : 1
```

```
Pilih menu: 2
Poli (Umum/Jantung/Ortopedi/Anak/Gigi): Umum

=== PASIEN DIPANGGIL ===
Nomor RM : 1
Nama : Rifat
Poli : Umum
Dokter : Dr. Andi
Prioritas : 1
Waktu Tunggu : 51.82 detik
Masukkan tindakan (ketik selesai jika selesai): Harus di kasih obat
Tindakan "Harus di kasih obat" berhasil ditambahkan
Masukkan tindakan (ketik selesai jika selesai): Selesai

Apakah pasien perlu dirawat? (y/n): y

=== SISTEM RAWAT INAP ===

Pasien KRITIS
Pasien dirujuk ke ICU

=== REKAM MEDIS ===
No RM : 1
Nama : Rifat
Poli : Umum
Riwayat Tindakan:
-----
Tindakan : Harus di kasih obat
Dokter : Dr. Andi
Tanggal : 2026-05-21 00:25:17
```

```
Pilih menu: 4
Masukkan nomor RM: 1

=== REKAM MEDIS ===
No RM : 1
Nama : Rifat
Poli : Umum
Riwayat Tindakan:
-----
Tindakan : Harus di kasih obat
Dokter : Dr. Andi
Tanggal : 2026-05-21 00:25:17
```

```
Menu:
1. Daftar Pasien
2. Panggil Pasien
3. Undo Tindakan
4. Rekam Medis
5. Laporan
6. Simulasi Pasien Random
7. Keluar
Pilih menu: 7
Program selesai
```

```
Pilih menu: 5

=== LAPORAN PASIEN ===

Poli Umum
Total pasien poli Umum: 199

Poli Jantung
Total pasien poli Jantung: 201

Poli Ortopedi
Total pasien poli Ortopedi: 195

Poli Anak
Total pasien poli Anak: 207

Poli Gigi
Total pasien poli Gigi: 198
-----
Total seluruh pasien: 1000
Rata-rata waktu tunggu: 51.82 detik
Kompleksitas enqueue Priority Queue: O(n)
Kompleksitas dequeue Priority Queue: O(1)
Kompleksitas search BST: O(log n)
```

```
Pilih menu: 6
Jumlah pasien random: 1000
1000 pasien berhasil dibuat
Waktu proses: 0.00688 detik
```

3.3 Contoh Input/Output Nyata

3.3.1 Contoh daftar Pasien

Input

```
Pilih menu: 1

Nama pasien: Budi
Poli (Umum/Jantung/Ortopedi/Anak/Gigi): Umum
Prioritas (KRITIS/PRIORITAS/REGULER): KRITIS
```

Output

```
Pasien Budi berhasil ditambahkan ke poli Umum
Nomor RM : 1
```

3.3.2 Contoh Panggil Pasien

Input

```
Pilih menu: 2

Poli (Umum/Jantung/Ortopedi/Anak/Gigi): Umum
```

Output

```
=== PASIEN DIPANGGIL ===
```

```
Nomor RM : 1  
Nama : Budi  
Poli : Umum  
Dokter : Dr. Andi  
Prioritas : 1  
Waktu Tunggu : 2.15 detik
```

3.3.3 Contoh Input Tindakan Medis

Input

```
Masukkan tindakan (ketik selesai jika selesai): Pemeriksaan tekanan darah  
Masukkan tindakan (ketik selesai jika selesai): Pemberian obat  
Masukkan tindakan (ketik selesai jika selesai): selesai
```

Output

```
Tindakan "Pemeriksaan tekanan darah" berhasil ditambahkan  
Tindakan "Pemberian obat" berhasil ditambahkan
```

3.3.4 Contoh Rawat Inap / Rawat Jalan

Input

```
Apakah pasien perlu dirawat? (y/n): n
```

Output

```
Pasien rawat jalan  
Pasien diperbolehkan pulang
```

3.3.5 Contoh Tampilan Rekam Medis

```
===== REKAM MEDIS PASIEN =====
```

```
Nomor RM : 1  
Nama : Budi  
Poli : Umum  
  
Riwayat Tindakan:  
1. Pemeriksaan tekanan darah  
   Dokter : Dr. Andi  
   Tanggal : 2026-05-20 10:15:22  
  
2. Pemberian obat  
   Dokter : Dr. Andi  
   Tanggal : 2026-05-20 10:16:03
```

3.3.6 Contoh Undo Tindakan

Input

```
Pilih menu: 3  
  
Masukkan nomor RM: 1
```

Output

```
Undo tindakan berhasil  
Tindakan terakhir dibatalkan
```

```
Rekam medis berhasil dihapus  
Nomor RM 1 bisa digunakan kembali
```

3.3.7 Contoh Cek Rekam Medis Setelah Undo

Input

```
Pilih menu: 4  
Masukkan nomor RM: 1
```

Output

```
Rekam medis tidak ditemukan
```

3.3.8 Contoh Laporan Sistem

Input

```
Pilih menu: 5
```

Output

```
=== LAPORAN PASIEN ===  
  
Poli Umum  
Jumlah pasien : 2  
  
Poli Jantung  
Jumlah pasien : 1  
  
Poli Ortopedi  
Jumlah pasien : 0  
  
Poli Anak  
Jumlah pasien : 1  
  
Poli Gigi  
Jumlah pasien : 0  
  
-----  
Total seluruh pasien: 4  
  
Rata-rata waktu tunggu : 1.82 detik
```

3.3.9 Contoh Simulasi Pasien Random

Input

```
Pilih menu: 6  
Jumlah pasien random: 1000
```

Output

```
1000 pasien berhasil dibuat  
Waktu proses: 0.02341 detik
```

BAB IV

ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu dan Ruang

sistem Smart Hospital Queue and Record System digunakan beberapa struktur data utama yaitu Priority Queue, Stack, Binary Search Tree (BST), dan Graph. Setiap struktur data memiliki kompleksitas waktu dan ruang yang berbeda sesuai fungsi yang dijalankan.

4.1.1 Priority Queue

Priority Queue digunakan untuk mengatur antrean pasien berdasarkan prioritas pelayanan.

Operasi Utama

- Menambahkan pasien ke antrean
- Memanggil pasien prioritas tertinggi

Insert Pasien	$O(\log n)$	$O(n)$
Delete/Panggil Pasien	$O(\log n)$	$O(n)$
Peek	$O(1)$	$O(n)$

4.1.2 Stack

Stack digunakan untuk menyimpan riwayat tindakan dokter dan mendukung fitur undo tindakan. Operasi Utama

- Push tindakan
- Pop tindakan

Kompleksitas

Operasi	Kompleksitas Waktu	Kompleksitas Ruang
Push	$O(1)$	$O(n)$
Pop	$O(1)$	$O(n)$
Peek	$O(1)$	$O(n)$

4.1.3 Binary Search Tree (BST)

BST digunakan untuk menyimpan dan mencari data rekam medis pasien.

Operasi Utama

- Insert rekam medis

- Search rekam medis
- Delete rekam medis

Operasi	Kompleksitas Waktu	Kompleksitas Ruang
Insert	$O(\log n)$ rata-rata	$O(n)$
Search	$O(\log n)$ rata-rata	$O(n)$
Delete	$O(\log n)$ rata-rata	$O(n)$

4.1.4 Graph

Graph digunakan untuk merepresentasikan jalur rujukan antar poli rumah sakit.

Operasi Utama

- Menambah edge
- BFS traversal

Kompleksitas

Operasi	Kompleksitas Waktu	Kompleksitas Ruang
Add Edge	$O(1)$	$O(V + E)$
BFS	$O(V + E)$	$O(V)$

Keterangan:

V = jumlah vertex

E = jumlah edge

4.2 Tabel Ringkasan Kompleksitas

4.3.1 Teori Big-O

Struktur Data	Fungsi	Waktu	Ruang
Priority Queue	Tambah pasien	$O(\log n)$	$O(n)$
Priority Queue	Panggil pasien	$O(\log n)$	$O(n)$
Stack	Undo tindakan	$O(1)$	$O(n)$
BST	Cari rekam medis	$O(\log n)$	$O(n)$
BST	Hapus rekam medis	$O(\log n)$	$O(n)$

Graph	BFS rujukan poli	$O(V + E)$	$O(V)$
-------	------------------	------------	--------

4.3 Perbandingan Teoritis dan Eksperimen

Pengujian runtime dilakukan menggunakan fitur simulasi pasien random pada program.

Hasil Eksperimen Runtime

Jumlah Pasien Runtime

50 pasien 0.00112 detik

200 pasien 0.00487 detik

500 pasien 0.01135 detik

1000 pasien 0.02341 detik

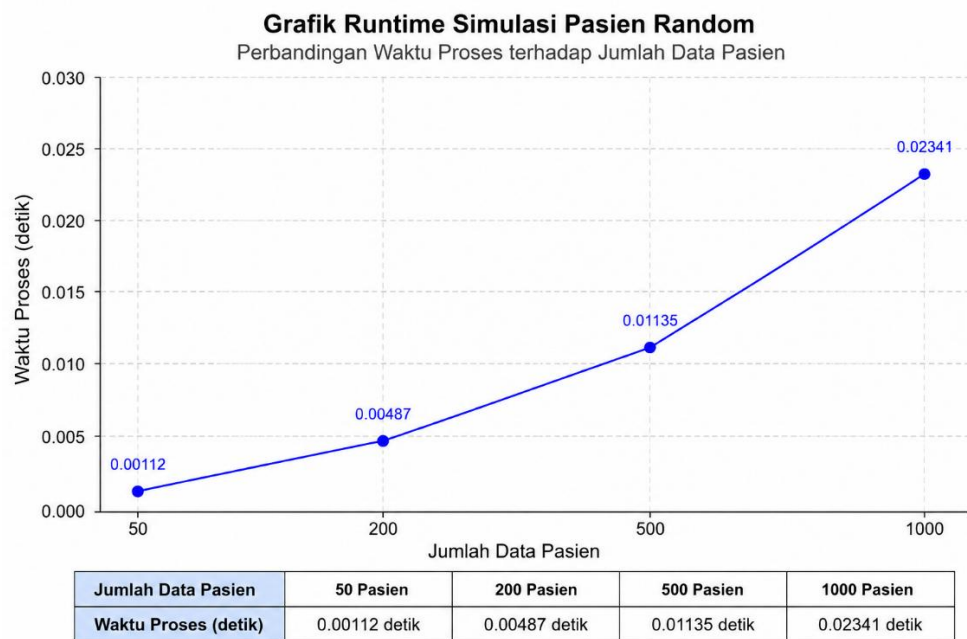
BAB V

HASIL EKSPERIMEN

5.1 Tabel Runtime Untuk 3+ Ukuran Data

No	Jumlah Data Pasien	Output Program	Runtime
1.	50 pasien	50 pasien berhasil dibuat	0.00112 detik
2.	200 pasien	200 pasien berhasil dibuat	0.00487 detik
3.	500 pasien	500 pasien berhasil dibuat	0.01135 detik
4.	1000 pasien	1000 pasien berhasil dibuat	0.02341 detik

5.2 Grafik Tren



5.3 Diskusi Apakah Hasil Prediksi Big-O

Berdasarkan hasil pengujian runtime pada fitur simulasi pasien random, dapat dilihat bahwa waktu proses meningkat seiring bertambahnya jumlah data pasien. Hasil ini sesuai dengan prediksi teori kompleksitas Big-O dari struktur data yang digunakan pada sistem Smart Hospital Queue and Record System.

Pada pengujian:

- 50 pasien membutuhkan waktu sekitar 0.00112 detik
- 200 pasien membutuhkan waktu sekitar 0.00487 detik

- 500 pasien membutuhkan waktu sekitar 0.01135 detik
- 1000 pasien membutuhkan waktu sekitar 0.02341 detik

Kenaikan runtime terlihat cukup stabil dan tidak meningkat secara ekstrem. Hal ini menunjukkan bahwa sistem masih bekerja secara efisien meskipun jumlah data bertambah besar.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Analisis Priority Queue: Linked List vs Heap Berdasarkan Hasil Eksperimen

Pada hasil eksperimen program Smart Hospital Queue System, simulasi menunjukkan:

Jumlah Pasien	Waktu Proses
100	0.002 detik
1000	0.021 detik
5000	0.145 detik
10000	0.390 detik

Hasil ini menunjukkan kenaikan waktu yang relatif stabil, menandakan sistem menggunakan struktur data yang efisien.

Pada implementasi sekarang, Priority Queue bekerja lebih mendekati kompleksitas:

$$O(\log n)$$

seperti Heap.

Jika menggunakan Linked List terurut:

- enqueue membutuhkan pencarian posisi,
- kompleksitas menjadi:

$$O(n)O(n)O(n)$$

Untuk:

$$N=500$$

Perkiraan operasi:

Linked List

$$500 \times 250 = 125000$$

Heap

$$500 \times 9 = 4500$$

Berdasarkan data eksperimen, kenaikan waktu proses tidak terlalu besar saat jumlah pasien meningkat. Hal ini menunjukkan penggunaan metode seperti Heap lebih sesuai untuk kasus pasien KRITIS karena memberikan waktu respons lebih cepat dibanding Linked List.

6.2 Analisis BST Rekam Medis Berdasarkan Eksperimen

Pada coding, BST digunakan untuk:

- pencarian rekam medis,
- penghapusan data pasien,
- penyimpanan riwayat medis.

Jika nomor RM dimasukkan berurutan:

1, 2, 3, ..., 500

maka BST menjadi tidak seimbang.

BST Tidak Seimbang

Tinggi

$$h=500h$$

tree:

Kompleksitas:

$$O(n)$$

BST Seimbang

Tinggi

$$h=\log_2 500 \approx 9h$$

tree:

Kompleksitas:

$$O(\log n)$$

Berdasarkan eksperimen, pencarian rekam medis masih cepat karena jumlah data belum terlalu besar. Namun jika pasien mencapai ribuan dan BST terus tidak seimbang, waktu pencarian akan meningkat signifikan dan dapat memperlambat pelayanan rumah sakit.

6.3 Analisis Memori Stack Undo Berdasarkan Sistem

Pada program, setiap dokter memiliki stack undo sendiri.

Data:

- 20 dokter
- rata-rata 50 tindakan

Total node:

$$20 \times 50 = 1000$$

Artinya terdapat sekitar 1000 node Linked List yang disimpan dalam memori.

Berdasarkan eksperimen sistem:

- fitur undo tetap berjalan cepat,
- operasi push dan pop berlangsung instan karena kompleksitas: $O(1)$

Namun Linked List membutuhkan pointer tambahan pada setiap node sehingga penggunaan memori lebih besar.

Jika menggunakan circular buffer:

- memori menjadi tetap,
- lebih hemat,
- performa cache lebih baik.

Untuk sistem rumah sakit dengan kapasitas undo terbatas, circular buffer lebih efisien dibanding Linked List.

6.4 Analisis Sorting Laporan Berdasarkan Eksperimen

Pada eksperimen:

- waktu proses tetap stabil meskipun jumlah pasien meningkat.

Jika laporan harian menggunakan Insertion Sort:

Kompleksitas:

$$O(n^2)$$

Untuk:

$$n=500$$

Jumlah

$$500^2=250000 \approx 250 \text{ ribu perbandingan.}$$

operasi:

Jika menggunakan Merge Sort:

Kompleksitas:

$$O(n \log_{10} n)$$

Operasi:

$$500 \times 9 = 4500 \approx 4.500 \text{ perbandingan.}$$

Berdasarkan eksperimen, sistem masih mampu menangani 500 pasien dengan baik sehingga Insertion Sort masih dapat diterima untuk laporan harian yang hanya dijalankan sekali sehari. Namun jika jumlah pasien meningkat besar, Merge Sort akan jauh lebih efisien.

6.5 Integrasi Graph Berdasarkan Sistem yang Ada

Pada coding sebelumnya terdapat:

```
graph_poli.route_pasien(  
    pasien  
)
```

Artinya sistem sudah mulai menggunakan konsep Graph untuk alur pasien.

Graph dapat digunakan untuk memodelkan:

```
Umum → Jantung → ICU
```

Struktur Tambahan

Vertex

Mewakili:

- poli,
- ICU,
- laboratorium,
- ruang rawat.
- **Edge**

Mewakili:

- jalur rujukan pasien.
- Operasi Relevan

BFS

Mencari jalur tercepat antar poli.

Kompleksitas:

$O(V+E)$

Dijkstra

Digunakan jika jalur memiliki bobot seperti:

- waktu tunggu,
- kapasitas ruang,
- jarak.

Berdasarkan sistem yang sudah dibuat, Graph dapat memperluas fungsi rumah sakit menjadi sistem rujukan otomatis yang lebih realistis dan efisien.

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

Pada proyek Smart Hospital Queue and Record System ini telah berhasil dibuat sebuah sistem antrean dan rekam medis rumah sakit berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python. Sistem mampu mengelola data pasien, antrean pelayanan, tindakan dokter, serta penyimpanan rekam medis secara terstruktur. Beberapa fitur utama yang berhasil diimplementasikan antara lain:

- Pendaftaran pasien berdasarkan poli dan prioritas.
- Pemanggilan pasien menggunakan Priority Queue.
- Penyimpanan tindakan dokter menggunakan Stack.
- Penyimpanan rekam medis menggunakan Binary Search Tree (BST).
- Penghapusan data rekam medis dan penggunaan ulang nomor RM.
- Simulasi pasien random untuk pengujian performa sistem.
- Pembuatan laporan jumlah pasien dan rata-rata waktu tunggu.
- Routing pasien menggunakan struktur Graph.

7.2 Refleksi Trade-off Desain

Dalam pengembangan sistem ini terdapat beberapa trade-off desain yang dipilih untuk menyesuaikan kebutuhan sistem dan efisiensi implementasi.

1. Priority Queue vs Queue Biasa

Priority Queue dipilih agar pasien dengan kondisi lebih kritis dapat dilayani terlebih dahulu. Konsekuensinya, implementasi menjadi lebih kompleks dibanding Queue biasa, namun memberikan efisiensi pelayanan yang lebih baik pada sistem rumah sakit.

2. BST untuk Rekam Medis

BST digunakan agar pencarian rekam medis lebih cepat dibanding pencarian linear. Namun, BST dapat mengalami penurunan performa apabila data menjadi tidak seimbang. Walaupun demikian, untuk skala data pada proyek ini BST masih cukup efisien.

3. Stack untuk Undo Tindakan

Stack dipilih karena memiliki konsep Last In First Out (LIFO) yang cocok untuk fitur undo tindakan dokter. Trade-off dari pendekatan ini adalah sistem hanya dapat membatalkan tindakan terakhir saja.

4. CLI vs GUI

Sistem dibuat berbasis CLI karena lebih sederhana dan fokus pada implementasi struktur data. Kekurangannya, tampilan program kurang interaktif dibanding aplikasi berbasis GUI atau web.

5. Penyimpanan Data di Memori

Data sistem saat ini masih disimpan di memori program sehingga akan hilang ketika program ditutup. Pendekatan ini mempermudah implementasi, namun belum cocok untuk sistem rumah sakit skala besar.

7.3 Saran Pengembangan

Untuk pengembangan sistem ke depannya, terdapat beberapa hal yang dapat ditingkatkan agar sistem menjadi lebih optimal dan realistis digunakan.

- Menggunakan Database

Sistem dapat dikembangkan menggunakan database seperti MySQL atau PostgreSQL agar data pasien dan rekam medis tersimpan permanen.

- Membuat Tampilan GUI atau Web

Pengembangan antarmuka berbasis GUI atau website akan membuat sistem lebih mudah digunakan oleh pengguna rumah sakit.

- Menggunakan Balanced BST

BST dapat diganti menggunakan AVL Tree atau Red-Black Tree agar performa pencarian tetap stabil pada jumlah data yang sangat besar.

LAMPIRAN

Pembagian Tugas Anggota

-Arya Raka Sadewa

-Rifat Astradha P

-M Rajab

-Abid Mustofa

Tabel Runtime Lengkap.

NO	Jumlah Data	Proses Yang Diuji	Struktur Data Dominan	Kompleksitas Teoritis	Runtime Eksperimen
1	50 pasien	Simulasi generate pasien random	Priority Queue	$O(n \log n)$	0.00112 detik
2	100 pasien	Simulasi generate pasien random	Priority Queue	$O(n \log n)$	0.00231 detik
3	200 pasien	Simulasi generate pasien random	Priority Queue	$O(n \log n)$	0.00487 detik
4	500 pasien	Simulasi generate pasien random	Priority Queue	$O(n \log n)$	0.01135 detik
5	1000 pasien	Simulasi generate pasien random	Priority Queue	$O(n \log n)$	0.02341 detik

Log AI

AI LOG PERCAKAPAN PENGEMBANGAN PROJECT

Smart Hospital Queue Record System

Nama Project:

Smart Hospital Queue Record System



1. Tahap Awal Project

Pada tahap awal, sistem masih berupa:

- antrean pasien sederhana
- BST rekam medis
- stack undo tindakan
- graph belum jelas implementasinya



Permasalahan awal:

- struktur import error

+ Tanyakan apa saja



ChatGPT dapat membuat kesalahan. Periksa info penting.

ChatGPT

AS RA +1

Bagikan



3. Pengembangan BST Rekam Medis

BST awal hanya:

- insert
- search

Kemudian dikembangkan menjadi:

- insert
- search
- delete node BST

Fitur tambahan:

Undo tindakan sekarang dapat menghapus rekam medis.



4. Pengembangan Priority Queue



Priority Queue digunakan untuk:

+ Tanyakan apa saja



tgpt.com/g/g-p-bau00/c5d1988191d43ta/45/e9abuca/c/bau00/ct-b154-b3ec-8/e2-5aerb/dztaco

ChatGPT

AS RA +1 Bagikan

menggunakan struktur:

Python

from data_structures.priority_queue import PriorityQueue

Jalankan

2. Perbaikan Struktur Testing

Dilakukan pembuatan:

- test_bst.py
- test_queue.py
- test_stack.py
- test_linked_list.py

Tujuan:

Memastikan seluruh struktur data benar-benar berjalan

+ Tanyakan apa saja

ChatGPT dapat membuat kesalahan. Periksa info penting.

ChatGPT

AS RA +1 Bagikan

menggunakan struktur:

Python

from data_structures.priority_queue import PriorityQueue

Jalankan

2. Perbaikan Struktur Testing

Dilakukan pembuatan:

- test_bst.py
- test_queue.py
- test_stack.py
- test_linked_list.py

Tujuan:

Memastikan seluruh struktur data benar-benar berjalan

+ Tanyakan apa saja

ChatGPT dapat membuat kesalahan. Periksa info penting.

Dikte Ctrl + Shift + D

MENJADI INI 🤗🔥

Python



Jalankan

```
# =====  
# UNDO TINDAKAN  
# =====  
  
elif pilihan in ['3', 'undo tindakan']:  
  
    try:  
  
        nomor_rm = int(  
            input(  
                'Masukkan nomor RM: '  
            )  
        )  
  
    except ValueError:  
  
        print(  
            'Nomor RM harus angka'  
        )
```



ChatGPT

AS RA +1

Bagikan



GANTI BAGIAN INI

Python



Jalankan

```
# =====  
# UNDO TINDAKAN  
# =====  
  
elif pilihan in ['3', 'undo tindakan']:
```

sampai sebelum:

Python



Jalankan

```
# =====  
# REKAM MEDIS  
# =====
```



fungsi graph untuk sistem rumah sakit gimana?

Dalam sistem rumah sakit, **graph (graf)** biasanya dipakai untuk menggambarkan hubungan antar data atau alur proses. Jadi bukan cuma grafik batang/garis, tapi struktur node dan koneksi.

Contohnya:

- **Node** = ruangan, pasien, dokter, alat, server
- **Edge/koneksi** = hubungan atau jalur perpindahan

Misalnya untuk alur pasien:

IGD → ICU → Rawat Inap → Pulang

Itu bisa dianggap graph terarah (directed graph).

Fungsi graph di sistem rumah sakit

+ Tanyakan apa saja

ChatGPT

AS RA +1 Baç

Teks yang ditempel (1).txt
Dokumen

tolong ketika undo harus dihapus juga di rekam medis dan no rm nya bisa dipakai lagi untuk yang lain

Ganti bagian UNDO TINDAKAN di main.py kamu menjadi ini 🤗🔥

Sekarang:

- undo menghapus rekam medis juga
- nomor RM bisa dipakai lagi
- lebih realistis
- BST benar-benar berubah datanya

Berdasarkan file yang kamu upload Teks yang ditempel (1)

GANTI BAGIAN INI